Constraints (besides Primary and Foreign Keys)

```
ALTER TABLE user_responses
    ADD CONSTRAINT chk_relImp     CHECK (relImp     IS NULL OR relImp
BETWEEN 1 AND 11),
    ADD CONSTRAINT chk_sizeImp    CHECK (sizeImp    IS NULL OR
sizeImp    BETWEEN 1 AND 11),
    ADD CONSTRAINT chk_settingImp CHECK (settingImp IS NULL OR
settingImp BETWEEN 1 AND 11),
    ADD CONSTRAINT chk_regionImp  CHECK (regionImp  IS NULL OR
regionImp  BETWEEN 1 AND 11),
    ADD CONSTRAINT chk_stateImp   CHECK (stateImp   IS NULL OR
stateImp   BETWEEN 1 AND 11),
    ADD CONSTRAINT chk_satMath    CHECK (satMath    IS NULL OR
satMath    BETWEEN 200 AND 800),
    ADD CONSTRAINT chk_satEng     CHECK (satEng     IS NULL OR satEng
BETWEEN 200 AND 800),
    ADD CONSTRAINT chk_act        CHECK (act        IS NULL OR act
BETWEEN 1  AND 36),
    ADD CONSTRAINT chk_income     CHECK (income     IS NULL OR income
BETWEEN 0  AND 5);

ALTER TABLE colleges_dynamic
    ADD CONSTRAINT chk_admission_rate  CHECK (admission_rate   IS
NULL OR admission_rate  BETWEEN 0 AND 1),
    ADD CONSTRAINT chk_graduation_rate CHECK (graduation_rate  IS
NULL OR graduation_rate BETWEEN 0 AND 1);
```

Trigger

Event: AFTER INSERT on user_accounts
Condition: IF email domain is non-empty (real signup, not a system insert)
Action 1: INSERT row into audit_log
Action 2: UPDATE user_responses.signed_up_at

```
DROP TRIGGER IF EXISTS trg_after_user_signup;

DELIMITER $$

CREATE TRIGGER trg_after_user_signup
AFTER INSERT ON user_accounts
FOR EACH ROW
BEGIN
    DECLARE domain VARCHAR(255);
    SET domain = SUBSTRING_INDEX(NEW.email, '@', -1);

    IF domain != '' AND domain IS NOT NULL THEN
        INSERT INTO audit_log (event_type, table_name, record_id,
detail)
        VALUES (
            'SIGNUP',
            'user_accounts',
            NEW.id,
            CONCAT('New user: ', NEW.name, ' (', NEW.email, ')')
        );

        UPDATE user_responses
        SET signed_up_at = NOW()
        WHERE id = NEW.id;
    END IF;
END$$

DELIMITER ;
```

Stored Procedure

GetTopColleges(p_response_id)

Returns candidate colleges after applying hard filters from user preferences.
Scoring is left to output.py. Two advanced queries are used:

- Query 1 (preparing table for filtering out by majors). Features: JOIN, GROUP BY/HAVING, subquery.
- Query 2 (hard filters for majors, state, region, special preferences). Features: JOIN,

subquery.

```
DROP PROCEDURE IF EXISTS GetTopColleges;

DELIMITER $$

CREATE PROCEDURE GetTopColleges(IN p_response_id INT)
BEGIN
    DECLARE v_regImp    INT DEFAULT 0;
    DECLARE v_stImp     INT DEFAULT 0;
    DECLARE v_relImp    INT DEFAULT 0;
    DECLARE v_sizeImp   INT DEFAULT 0;
    DECLARE v_setImp    INT DEFAULT 0;
    DECLARE v_allMajors BOOLEAN DEFAULT FALSE;
    DECLARE v_has_majors INT DEFAULT 0;

    SELECT regionImp, stateImp, relImp, sizeImp, settingImp,
allMajors
    INTO   v_regImp, v_stImp, v_relImp, v_sizeImp, v_setImp,
v_allMajors
    FROM   user_responses
    WHERE  id = p_response_id;

    -- Check whether the user selected any majors at all
    SELECT COUNT(*) INTO v_has_majors
    FROM user_majors WHERE user_id = p_response_id;

    -- Query 1: Major filter

    DROP TEMPORARY TABLE IF EXISTS tmp_major_colleges;
    CREATE TEMPORARY TABLE tmp_major_colleges AS
        SELECT cm.college_id,
               COUNT(DISTINCT cm.major_id) AS matched_major_count
        FROM college_majors cm
        JOIN user_majors um
            ON cm.major_id = um.code
            AND um.user_id = p_response_id
        GROUP BY cm.college_id
```

```
        HAVING (
            v_allMajors = FALSE
            OR v_allMajors IS NULL
            OR COUNT(DISTINCT cm.major_id) = (
                SELECT COUNT(DISTINCT code)
                FROM user_majors
                WHERE user_id = p_response_id
            )
        );

    -- Query 2: Hard filters + final result

    SELECT
        cs.college_id,
        cs.name,
        cs.website,
        cs.finaid_website,
        cs.city,
        cs.state,
        cs.locale,
        cs.region,
        cs.type,
        cs.religious_affiliation,
        cs.online_only,
        cs.hbcu,
        cs.annh,
        cs.aanipi,
        cs.tribal,
        cs.hispanic,
        cs.men_only,
        cs.women_only,
        cd.admission_rate,
        cd.sat_rw_mid,
        cd.sat_math_mid,
        cd.sat_avg,
        cd.act_avg,
        cd.graduation_rate,
        cd.median_earnings_6yr,
        cd.median_earnings_10yr,
```

```
        cd.avg_cost_of_attendance,
        cd.in_state_tuition_fees,
        cd.out_state_tuition_fees,
        cd.net_price_0_30k,
        cd.net_price_30_48k,
        cd.net_price_48_75k,
        cd.net_price_75_110k,
        cd.net_price_110k_plus,
        cd.median_starting_debt,
        cd.num_students,
        COALESCE(tmc.matched_major_count, 0) AS matched_major_count
    FROM colleges_static cs
    JOIN colleges_dynamic cd
        ON cs.college_id = cd.college_id
   -- Most recent year
        AND cd.year = (
            SELECT MAX(cd2.year)
            FROM colleges_dynamic cd2
            WHERE cd2.college_id = cs.college_id
        )
    -- Only include colleges that passed the major filter
    -- (skip join entirely if user selected no majors)
    LEFT JOIN tmp_major_colleges tmc
        ON cs.college_id = tmc.college_id
    WHERE
        -- Major filter: skip if no majors selected, otherwise
require a match
        (v_has_majors = 0 OR tmc.college_id IS NOT NULL)
        -- Hard filter: state (only when stateImp = 11)
        AND (v_stImp < 11 OR cs.state IN (
            SELECT name FROM states WHERE user_id = p_response_id
        ))
        -- Hard filter: region (only when regionImp = 11)
        AND (v_regImp < 11 OR cs.region IN (
            SELECT code - 1 FROM regions WHERE user_id =
p_response_id
        ))
        -- Hard filters: spec prefs (NOT EXISTS for each)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
```

```
p_response_id AND code = 1) OR cs.hbcu      = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 2) OR cs.annh      = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 3) OR cs.aanipi    = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 4) OR cs.hispanic  = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 5) OR cs.tribal    = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 6) OR cs.men_only  = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 7) OR cs.women_only = 1)
        AND (NOT EXISTS (SELECT 1 FROM specPrefs WHERE user_id =
p_response_id AND code = 8) OR cs.online_only = 1);

    DROP TEMPORARY TABLE IF EXISTS tmp_major_colleges;

END$$

DELIMITER ;
```

Transaction

The transaction is wrapped in another stored procedure, SaveUserPreferences. The SERIALIZABLE transaction validates preferences, then atomically deletes old rows and updates scalars. Two advanced queries are used:

- Query 1 (contradiction check to prevent simultaneous selection of men_only and women_only). Features: JOIN, GROUP BY/HAVING, subquery.
- Query 2 (audit preferences and insert into audit table before deleting them). Features: JOIN, set operations (UNION), subquery.

```
DROP PROCEDURE IF EXISTS SaveUserPreferences;

DELIMITER $$

CREATE PROCEDURE SaveUserPreferences(
    IN p_uid        INT,
    IN p_relImp     INT,
    IN p_sizeImp    INT,
    IN p_allMajors  BOOLEAN,
    IN p_satMath    INT,
    IN p_satEng     INT,
    IN p_act        INT,
    IN p_settingImp INT,
    IN p_regionImp  INT,
    IN p_stateImp   INT,
    IN p_income     INT
)
BEGIN
    DECLARE v_conflict  INT DEFAULT 0;
    DECLARE v_pref_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    -- Query 1: Contradiction check on previous data (it's about to
get replaced)

    SELECT COUNT(*) INTO v_conflict
    FROM (
        SELECT sp1.user_id
        FROM specPrefs sp1
        JOIN specPrefs sp2
            ON sp1.user_id = sp2.user_id
            AND sp1.user_id = p_uid
        WHERE sp1.code = 6
```

```
        AND sp2.code = 7
        GROUP BY sp1.user_id
        HAVING COUNT(DISTINCT sp1.code) + COUNT(DISTINCT sp2.code) >=
2
    ) AS conflict_check;

    IF v_conflict > 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Cannot select both men-only and
women-only preferences.';
    END IF;

    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    START TRANSACTION;
-
        -- Query 2: Before deleting, audit with LEFT JOIN + UNION

        INSERT INTO audit_log (event_type, table_name, record_id,
detail)
        SELECT
            'PREF_UPDATE',
            'user_responses',
            ur.id,
            CONCAT(
                'Preferences about to be updated for ',
                COALESCE(ua.name, 'anonymous'),
                '. Total preferences previously selected: ',
                (
                    SELECT COUNT(*) FROM (
                        SELECT id FROM religions  WHERE user_id =
p_uid
                        UNION ALL
                        SELECT id FROM sizes       WHERE user_id =
p_uid
                        UNION ALL
                        SELECT id FROM user_majors WHERE user_id =
p_uid
                        UNION ALL
                        SELECT id FROM settings    WHERE user_id =
```

```
p_uid
                    UNION ALL
                    SELECT id FROM regions     WHERE user_id =
p_uid
                    UNION ALL
                    SELECT id FROM states      WHERE user_id =
p_uid
                    UNION ALL
                    SELECT id FROM specPrefs   WHERE user_id =
p_uid
                ) AS all_prefs
            )
        )
    FROM user_responses ur
    LEFT JOIN user_accounts ua ON ua.id = ur.id
    WHERE ur.id = p_uid;

   -- DELETE all old multi-select entries, update scalars
    DELETE FROM religions   WHERE user_id = p_uid;
    DELETE FROM sizes       WHERE user_id = p_uid;
    DELETE FROM user_majors WHERE user_id = p_uid;
    DELETE FROM settings    WHERE user_id = p_uid;
    DELETE FROM regions     WHERE user_id = p_uid;
    DELETE FROM states      WHERE user_id = p_uid;
    DELETE FROM specPrefs   WHERE user_id = p_uid;

    UPDATE user_responses
    SET relImp     = p_relImp,
        sizeImp    = p_sizeImp,
        allMajors  = p_allMajors,
        satMath    = p_satMath,
        satEng     = p_satEng,
        act        = p_act,
        settingImp = p_settingImp,
        regionImp  = p_regionImp,
        stateImp   = p_stateImp,
        income     = p_income
    WHERE id = p_uid;
```

```
   COMMIT;
END$$

DELIMITER ;
```